DSA Patterns

Complete Interview Preparation Guide

20 Essential Coding Patterns

#	Pattern	Category	Time	Space
1	Sliding Window	Array / String	O(n)	O(1) or O(k)
2	Two Pointers	Array / Linked List	O(n)	O(1)
3	Binary Search	Array / Search Space	O(log n)	O(1)
4	Fast and Slow Pointers	Linked List / Cycle Detection	O(n)	O(1)
5	Merge Intervals	Array / Intervals	O(n log n)	O(n)
6	Top K Elements	Heap / Priority Queue	O(n log k)	O(k)
7	K-way Merge	Heap / Merge	O(n log k)	O(k)
8	Breadth-First Search (BFS)	Graph / Tree	O(V + E)	O(V)
9	Depth-First Search (DFS)	Graph / Tree	O(V + E)	O(V)
10	Backtracking	Recursion / Search	O(n!) worst case	O(n)
11	Dynamic Programming (DP)	Optimization	$O(n^2)$ typical	O(n) or $O(n^2)$
12	Kadane's Algorithm	Array / DP	O(n)	O(1)
13	Knapsack Problem	DP / Optimization	O(n × W)	O(n × W) or O(W) optimized
14	Tree Depth-First Search	Binary Tree	O(n)	O(h) — h = height
15	Tree Breadth-First Search	Binary Tree	O(n)	O(n)
16	Topological Sort	Graph / DAG	O(V + E)	O(V)
17	Trie	Data Structure / String	O(L) per op	O(N × L)
18	Graph — Bipartite Check	Graph / Coloring	O(V + E)	O(V)
19	Bitwise XOR	Bit Manipulation	O(n)	O(1)
20	Sliding Window — Optimal	Array / String (Advanced)	O(n)	O(k)

01. Sliding Window

Category: Array / String ■ **Time: O(n)** ■ **Space: O(1) or O(k)**

■ Description

Maintains a window (subarray/substring) that slides across the data structure. The window expands or shrinks based on conditions, avoiding redundant recomputation.

■ When to Use

Fixed or variable-size subarrays/substrings; problems asking for max/min/sum in contiguous range.

■ Classic Problems

• Maximum sum subarray of size k • Longest substring without repeating chars • Minimum window substring

■ Template

```
left = 0 for right in range(len(arr)): window.add(arr[right]) while window_invalid: window.remove(arr[left]) left += 1 update_result()
```

---

02. Two Pointers

Category: Array / Linked List ■ **Time: O(n)** ■ **Space: O(1)**

■ Description

Uses two indices moving toward each other or in the same direction to solve problems in O(n) instead of $O(n^2)$. Works on sorted arrays or linked lists.

■ When to Use

Pair/triplet sums, palindrome checks, removing duplicates from sorted arrays.

■ Classic Problems

• Two Sum II (sorted) • 3Sum • Container With Most Water • Valid Palindrome

■ Template

```
left, right = 0, len(arr) - 1 while left < right: if condition(arr[left], arr[right]): process() elif arr[left] + arr[right] < target: left += 1 else: right -= 1
```

---

03. Binary Search

Category: Array / Search Space ■ **Time: O(log n)** ■ **Space: O(1)**

■ Description

Divides the search space in half each iteration. Applicable beyond sorted arrays — any monotonic function or answer-space problem can use binary search.

■ When to Use

Sorted arrays, rotated arrays, finding boundaries, minimizing/maximizing over a search space.

■ Classic Problems

• Search in Rotated Sorted Array • Find Peak Element • Koko Eating Bananas

■ Template

```
lo, hi = 0, len(arr) - 1 while lo <= hi: mid = (lo + hi) // 2 if arr[mid] == target:
return mid elif arr[mid] < target: lo = mid + 1 else: hi = mid - 1
```

---

04. Fast and Slow Pointers

Category: Linked List / Cycle Detection ■ **Time: O(n)** ■ **Space: O(1)**

■ Description

Two pointers move at different speeds (Floyd's Tortoise & Hare). The fast pointer moves 2x; if a cycle exists they will meet. Also finds middle of linked list.

■ When to Use

Detecting cycles, finding middle node, finding start of a cycle.

■ Classic Problems

• Linked List Cycle • Find Cycle Start • Happy Number • Middle of Linked List

■ Template

```
slow = fast = head while fast and fast.next: slow = slow.next fast = fast.next.next if
slow == fast: # cycle detected break
```

---

05. Merge Intervals

Category: Array / Intervals ■ **Time: O(n log n)** ■ **Space: O(n)**

■ Description

Sort intervals by start time, then greedily merge overlapping ones. A powerful pattern for scheduling, calendar, and range problems.

■ When to Use

Overlapping intervals, meeting rooms, insert interval, employee free time.

■ Classic Problems

• Merge Intervals • Insert Interval • Meeting Rooms II

■ Template

```
intervals.sort(key=lambda x: x[0]) merged = [intervals[0]] for start, end in intervals[1:]: if start <= merged[-1][1]: merged[-1][1] = max(merged[-1][1], end) else: merged.append([start, end])
```

---

06. Top K Elements

Category: Heap / Priority Queue ■ **Time: O(n log k)** ■ **Space: O(k)**

■ Description

Uses a min-heap of size K to efficiently track the K largest/smallest/most-frequent elements without full sorting.

■ When to Use

K largest/smallest, K most frequent, K closest points.

■ Classic Problems

• Kth Largest Element • Top K Frequent Elements • K Closest Points to Origin

■ Template

```
import heapq heap = [] for num in nums: heapq.heappush(heap, num) if len(heap) > k: heapq.heappop(heap) # heap[0] is the Kth largest
```

---

07. K-way Merge

Category: Heap / Merge ■ **Time: O(n log k)** ■ **Space: O(k)**

■ Description

Merges K sorted lists efficiently using a min-heap. Always extract the global minimum and push the next element from that list.

■ When to Use

Merging K sorted arrays/lists, smallest range covering K lists.

■ Classic Problems

• Merge K Sorted Lists • Kth Smallest in M Sorted Arrays • Smallest Range Covering K Lists

■ Template

```
heap = [(lists[i][0], i, 0) for i in range(len(lists))] heapq.heapify(heap) while heap:
val, i, j = heapq.heappop(heap) result.append(val) if j + 1 < len(lists[i]):
heapq.heappush(heap, (lists[i][j+1], i, j+1))
```

---

08. Breadth-First Search (BFS)

Category: Graph / Tree ■ **Time: O(V + E)** ■ **Space: O(V)**

■ Description

Explores all neighbors at the current depth before going deeper. Uses a queue. Guarantees shortest path in unweighted graphs.

■ When to Use

Shortest path, level-order traversal, connected components, word ladder.

■ Classic Problems

• Binary Tree Level Order • Word Ladder • Rotting Oranges • Walls and Gates

■ Template

```
from collections import deque queue = deque([start]) visited = {start} while queue:
node = queue.popleft() for neighbor in graph[node]: if neighbor not in visited:
visited.add(neighbor) queue.append(neighbor)
```

---

09. Depth-First Search (DFS)

Category: Graph / Tree ■ **Time: O(V + E)** ■ **Space: O(V)**

■ Description

Explores as deep as possible before backtracking. Can be implemented recursively or with an explicit stack. Used for path finding, cycle detection, and component analysis.

■ When to Use

Path existence, all paths, cycle detection, topological sort, islands.

■ Classic Problems

• Number of Islands • Clone Graph • Path Sum • Course Schedule

■ Template

```
def dfs(node, visited): if node in visited: return visited.add(node) process(node) for neighbor in graph[node]: dfs(neighbor, visited)
```

---

10. Backtracking

Category: Recursion / Search ■ **Time: O(n!) worst case** ■ **Space: O(n)**

■ Description

Builds solutions incrementally, abandoning partial solutions (pruning) as soon as they can't lead to a valid solution. Explores the solution space systematically.

■ When to Use

Permutations, combinations, subsets, N-Queens, Sudoku, word search.

■ Classic Problems

• Permutations • N-Queens • Combination Sum • Word Search • Sudoku Solver

■ Template

```
def backtrack(path, choices): if is_solution(path): results.append(path[:]) return for choice in choices: if is_valid(choice, path): path.append(choice) backtrack(path, remaining(choices, choice)) path.pop() # undo
```

---

11. Dynamic Programming (DP)

Category: Optimization ■ **Time: O(n²) typical** ■ **Space: O(n) or O(n²)**

■ Description

Breaks problems into overlapping subproblems and stores results (memoization/tabulation). Two approaches: top-down (recursive + memo) and bottom-up (iterative table).

■ When to Use

Optimal substructure + overlapping subproblems: counting ways, min/max cost, longest subsequence.

■ Classic Problems

• Fibonacci • Coin Change • Longest Common Subsequence • 0/1 Knapsack

■ Template

```
# Bottom-up tabulation dp = [0] * (n + 1) dp[0] = base_case for i in range(1, n + 1):
for j in range(i): dp[i] = max(dp[i], dp[j] + transition(j, i))
```

---

12. Kadane's Algorithm

Category: Array / DP ■ **Time: O(n)** ■ **Space: O(1)**

■ Description

Finds the maximum sum subarray in O(n). Tracks current subarray sum and resets when it goes negative. A classic DP-on-array pattern.

■ When to Use

Maximum subarray sum, maximum product subarray (with modifications).

■ Classic Problems

• Maximum Subarray • Maximum Product Subarray • Best Time to Buy and Sell Stock

■ Template

```
max_sum = curr_sum = nums[0] for num in nums[1:]: curr_sum = max(num, curr_sum + num)
max_sum = max(max_sum, curr_sum)
```

---

13. Knapsack Problem

Category: DP / Optimization ■ **Time: O(n × W)** ■ **Space: O(n × W) or O(W) optimized**

■ Description

0/1 Knapsack: each item used at most once. Unbounded Knapsack: items can repeat. Core DP pattern for resource-allocation and subset-sum problems.

■ When to Use

Partition problems, target sum, coin change, bounded/unbounded item selection.

■ Classic Problems

• 0/1 Knapsack • Partition Equal Subset Sum • Target Sum • Coin Change

■ Template

```
# 0/1 Knapsack dp = [[0] * (W + 1) for _ in range(n + 1)] for i in range(1, n + 1): for w in range(W + 1): dp[i][w] = dp[i-1][w] if weights[i-1] <= w: dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1])
```

---

14. Tree Depth-First Search

Category: Binary Tree ■ **Time: O(n)** ■ **Space: O(h) — h = height**

■ Description

Traverses trees via pre-order, in-order, or post-order DFS. Recursive approach naturally captures parent-child relationships and path information.

■ When to Use

Path sum, max depth, diameter, LCA, serialize/deserialize tree.

■ Classic Problems

• Path Sum II • Binary Tree Diameter • Max Path Sum • Lowest Common Ancestor

■ Template

```
def dfs(node, path, target): if not node: return path.append(node.val) if not node.left and not node.right: if sum(path) == target: result.append(path[:]) dfs(node.left, path, target) dfs(node.right, path, target) path.pop()
```

---

15. Tree Breadth-First Search

Category: Binary Tree ■ **Time: O(n)** ■ **Space: O(n)**

■ Description

Level-order traversal of trees using a queue. Processes all nodes level by level, useful for level-based properties and shortest paths in trees.

■ When to Use

Level order output, zigzag traversal, right/left side view, min depth.

■ Classic Problems

• Binary Tree Level Order Traversal • Right Side View • Zigzag Traversal • Min Depth

■ Template

```
queue = deque([root]) while queue: level_size = len(queue) level = [] for _ in range(level_size): node = queue.popleft() level.append(node.val) if node.left: queue.append(node.left) if node.right: queue.append(node.right) result.append(level)
```

---

16. Topological Sort

Category: Graph / DAG ■ **Time: O(V + E)** ■ **Space: O(V)**

■ Description

Linear ordering of vertices in a Directed Acyclic Graph (DAG) where for every edge u→v, u comes before v. Implemented via BFS (Kahn's) or DFS.

■ When to Use

Course prerequisites, task scheduling, build systems, dependency resolution.

■ Classic Problems

• Course Schedule I & II • Alien Dictionary • Task Scheduler

■ Template

```
# Kahn's Algorithm (BFS) in_degree = {v: 0 for v in graph} for u in graph: for v in graph[u]: in_degree[v] += 1 queue = deque([v for v in graph if in_degree[v] == 0]) while queue: node = queue.popleft() order.append(node) for neighbor in graph[node]: in_degree[neighbor] -= 1 if in_degree[neighbor] == 0: queue.append(neighbor)
```

---

17. Trie

Category: Data Structure / String ■ **Time: O(L) per op** ■ **Space: O(N × L)**

■ Description

A tree-like data structure for storing strings where each node represents a character. Enables O(L) insert/search where L is word length. Ideal for prefix-based operations.

■ When to Use

Autocomplete, spell check, word search in grid, prefix matching, IP routing.

■ Classic Problems

• Implement Trie • Word Search II • Replace Words • Design Search Autocomplete

■ Template

```
class TrieNode: def __init__(self): self.children = {} self.is_end = False class Trie: def __init__(self): self.root = TrieNode() def insert(self, word): node = self.root for ch in word: node = node.children.setdefault(ch, TrieNode()) node.is_end = True
```

---

18. Graph — Bipartite Check

Category: Graph / Coloring ■ **Time: O(V + E)** ■ **Space: O(V)**

■ Description

Determines if a graph can be 2-colored such that no two adjacent nodes share the same color. Uses BFS/DFS with alternating colors to detect odd-length cycles.

■ When to Use

Bipartite detection, graph coloring, 2-SAT, relationship conflict checking.

■ Classic Problems

• Is Graph Bipartite? • Possible Bipartition • Flower Planting With No Adjacent

■ Template

```
color = {} for start in range(n): if start in color: continue queue = deque([start]) color[start] = 0 while queue: node = queue.popleft() for nei in graph[node]: if nei not in color: color[nei] = 1 - color[node] queue.append(nei) elif color[nei] == color[node]: return False # not bipartite
```

---

19. Bitwise XOR

Category: Bit Manipulation ■ **Time: O(n)** ■ **Space: O(1)**

■ Description

XOR has key properties: a⊕a=0, a⊕0=a, commutative & associative. These make it powerful for finding single/missing numbers and toggling bits without extra space.

■ When to Use

Finding the single non-duplicate, missing number, complement, swapping without temp variable.

■ Classic Problems

• Single Number • Missing Number • Find Duplicate • XOR Queries of a Subarray

■ Template

```
# Find single number among pairs result = 0 for num in nums: result ^= num # all pairs cancel out return result # Missing number in [0..n] result = len(nums) for i, num in enumerate(nums): result ^= i ^ num
```

---

20. Sliding Window — Optimal

Category: Array / String (Advanced) ■ **Time: O(n)** ■ **Space: O(k)**

■ Description

An advanced variant that dynamically adjusts the window using hash maps or frequency arrays. Handles complex conditions like 'at most K distinct chars' or 'exact character frequency matching'.

■ When to Use

Substrings with constraints, permutation in string, minimum window with all characters.

■ Classic Problems

• Permutation in String • Find All Anagrams • Minimum Window Substring • Fruit Into Baskets

■ Template

```
from collections import defaultdict need = Counter(target) missing = len(need) left = res_left = 0 for right, ch in enumerate(s): if need[ch] > 0: missing -= 1 need[ch] -= 1 if missing == 0: # valid window while need[s[left]] < 0: need[s[left]] += 1 left += 1 # update result
```

Quick Reference Cheat Sheet

Array Patterns

1. Sliding Window	2. Two Pointers	3. Binary Search	5. Merge Intervals	6. Top K Elements	12. Kadane's Algorithm	20. Sliding Window — Optimal

Graph & Tree Patterns

8. Breadth-First Search (BFS)	9. Depth-First Search (DFS)	14. Tree Depth-First Search	15. Tree Breadth-First Search	16. Topological Sort	18. Graph — Bipartite Check

DP & Optimization

11. Dynamic Programming (DP)	12. Kadane's Algorithm	13. Knapsack Problem

Advanced Data Structures

7. K-way Merge	17. Trie

Search & Traversal

3. Binary Search	8. Breadth-First Search (BFS)	9. Depth-First Search (DFS)	10. Backtracking

Bit & Math Tricks

19. Bitwise XOR

Master these 20 patterns and you'll be ready for any coding interview!